

1. Arquitectura General del Proyecto

El sistema utilizará una **Arquitectura Basada en Componentes y Gestores (Manager Pattern)** operando bajo un **Game Loop** (Bucle de Juego) clásico, gobernado por una **Máquina de Estados Finitos (FSM)**.

- **Funcionamiento General:** El núcleo del juego (Game Engine) ejecutará un bucle constante que procesa eventos de entrada, actualiza la lógica matemática del juego (estado) y renderiza los gráficos en pantalla a 60 fotogramas por segundo (FPS).
- **Gestión de Estados:** La FSM controlará en qué "pantalla" se encuentra el jugador (Menú, Jugando, Pausa, Derrota). Solo el estado activo se actualizará y renderizará.
- **Interacción entre Componentes:** Para evitar el alto acoplamiento, los distintos sistemas (como Enemigos, Torres y Recursos) no se comunicarán directamente entre sí en todo momento. En su lugar, utilizarán el **Patrón Observador (Event Bus)** para emitir y escuchar eventos (ej. "Enemigo_Derrotado" -> El Gestor de Recursos escucha y añade oro).
- **Flujo del Videojuego:** Al iniciar, se cargan los recursos gráficos y de audio (Asset Loading) en memoria. La FSM inicia en el MenuState. Al seleccionar "Jugar", se transiciona al GameState, el cual inicializa el mapa, la base y los gestores de oleadas y construcción.

2. Estructura de Carpetas Recomendada

Una estructura profesional es vital para la mantenibilidad. Separaremos claramente el código fuente de los recursos (assets) y la configuración.

Plaintext

el_ultimo_bastion/

└─ assets/ # Recursos multimedia

— images/	# Sprites de pixel art/cartoon (torres, enemigos, UI)
— audio/	# Música de fondo y efectos de sonido (SFX)
— fonts/	# Fuentes tipográficas
— maps/	# Archivos de datos del mapa (ej. Tiled, JSON)
— config/	# Archivos de configuración general
— settings.json	# Resoluciones, volumen, atajos de teclado
— game_data.json	# Estadísticas de enemigos, costos de torres, vida del bastión
— docs/	# Documentación técnica y diagramas UML
— src/	# Código fuente principal
— core/	# Bucle principal, gestor de estados
— entities/	# Clases base, enemigos, torres, proyectiles
— managers/	# Gestores lógicos (oleadas, recursos, construcción)
— ui/	# Elementos de interfaz gráfica (botones, HUD)
— utils/	# Funciones auxiliares, matemáticas, colisiones
— world/	# Lógica del mapa, grilla de construcción, pathfinding
— tests/	# Pruebas unitarias y de integración
— .gitignore	# Archivos a ignorar por Git (ej. __pycache__)
— requirements.txt	# Dependencias del proyecto (ej. pygame-ce==2.3.2)
— main.py	# Punto de entrada de la aplicación

3. Módulos del Sistema

La división modular asegura que el trabajo pueda dividirse fácilmente y que los errores estén aislados.

Módulo	Función Principal
Core (Núcleo)	Contiene el inicializador de Pygame, el GameLoop y el StateManager (Gestor de Estados) que alterna entre menús y gameplay.
World (Mundo)	Administra la matriz del terreno (Grid), determina qué celdas son transitables o construibles, y maneja el algoritmo de búsqueda de rutas (Pathfinding, idealmente A* o Waypoints).
Entities (Entidades)	Define todos los objetos dinámicos del juego. Incluye herencia jerárquica: de una clase base Entidad derivan Enemigo, Estructura y Proyectoil.
Managers (Gestores)	Controlan el flujo de las mecánicas. WaveManager (genera enemigos según el tiempo), ResourceManager (controla oro/maná), y BuildManager (valida y ejecuta la colocación de torres).
Combat (Combate)	Módulo encargado de la detección de colisiones (hitboxes), cálculo de daño, rangos de ataque y aplicación de efectos (ej. ralentización).
UI (Interfaz)	Dibuja la información en pantalla (HUD) de manera independiente a la lógica del juego. Maneja clics en botones del menú y selección de estructuras.

4. Diseño de Clases

Para cumplir con los principios de POO, utilizaremos herencia para agrupar comportamientos comunes y composición para características específicas.

Clase Principal	Responsabilidad	Atributos Clave	Métodos Principales	Relaciones
GameManager	Orquestrar el flujo global de la partida y actualizar el resto de managers.	state, screen, clock, delta_time	run(), update(), draw(), change_state()	Contiene instancias de todos los <i>Managers</i> y la <i>FSM</i> .
Entity (Base)	Representar cualquier objeto dibujable con posición física en el juego.	x, y, image, rect	update(), render()	Clase padre de <i>Enemy</i> , <i>Tower</i> , <i>Projectile</i> .
Enemy (Hereda Entity)	Manejar el movimiento por la ruta y la recepción de daño.	hp, speed, damage, path_index	move(), take_damage(), reach_base()	Instanciado por <i>WaveManager</i> . Daña a <i>Bastion</i> .

Clase Principal	Responsabilidad	Atributos Clave	Métodos Principales	Relaciones
Tower (Hereda Entity)	Buscar objetivos, apuntar y disparar periódicamente.	range, damage, cooldown, target	find_target(), shoot(), upgrade()	Genera <i>Projectile</i> . Controlado por <i>BuildManager</i> .
WaveManager	Leer la configuración de oleadas y spawnear enemigos en intervalos precisos.	current_wave, spawn_timer, enemies_list	start_wave(), spawn_enemy(), check_wave_end()	Instancia <i>Enemy</i> . Notifica a <i>GameManager</i> .
Bastion	Representar el núcleo de la base y su vitalidad.	max_hp, current_hp	take_damage(), is_destroyed()	Escucha eventos del <i>Enemy</i> . Notifica a <i>GameManager</i> .

Clase Principal	Responsabilidad	Atributos Clave	Métodos Principales	Relaciones
Grid / Map	Manejar la lógica espacial para el movimiento y construcción.	cell_size, matrix, waypoints	is_buildable(), place_tower(), get_path()	Consultado por <i>Enemy</i> y <i>BuildManager</i> .

5. Diagrama Conceptual (Flujo del Sistema)

Plaintext

[INICIO DE LA APLICACIÓN]



(Módulo Core / main.py) -> Carga de Assets y Configuración



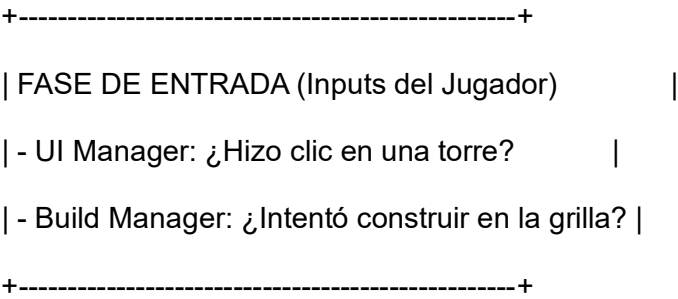
[ESTADO: MENÚ PRINCIPAL]

└─> Salir -> [FIN DE LA APLICACIÓN]

└─> Jugar -> Transición de Estado



[ESTADO: JUEGO (Game Loop a 60 FPS)]



↓

```

+-----+
| FASE DE ACTUALIZACIÓN (Lógica / Delta Time) |
| - Wave Manager: ¿Es hora de generar un Goblin? |
| - Enemigos: Moverse por el Pathfinding. |
| - Torres: Detectar enemigos en rango -> Disparar. |
| - Sistema de Combate: Detectar colisiones de |
| proyectiles -> Restar HP al enemigo. |
| - Bastion: ¿Llegó un enemigo? -> Restar Vida. |
+-----+

```

↓

```

+-----+
| FASE DE RENDERIZADO (Gráficos) |
| - Dibujar Mapa -> Torres -> Enemigos -> FX |
| - Dibujar UI (Vida del Bastión, Oro, Maná) |
+-----+

```

↓

[CONDICIONES DE FIN DE PARTIDA]

└─> Vida Bastión <= 0 -> [ESTADO: PANTALLA DE DERROTA]

└─> Oleadas superadas -> [ESTADO: PANTALLA DE VICTORIA]

6. Escalabilidad

Para asegurar que la investigación universitaria pueda evaluar la facilidad de extender el código (un pilar del software bien diseñado), la arquitectura contempla:

1. **Diseño Basado en Datos (Data-Driven):** Los atributos de enemigos (salud, velocidad de los Goblins) y torres no se codificarán en duro ("hardcoded") en las clases. Se leerán desde un archivo `game_data.json`. Agregar un nuevo enemigo

("Goblin Chamán") solo requerirá añadir un bloque de texto en el JSON y su sprite en la carpeta, sin tocar el código fuente.

2. **Patrón Factory:** Se utilizará un EnemyFactory y un TowerFactory para instanciar objetos. Si se añade una nueva mecánica (ej. Torre Mágica que ralentiza), la fábrica simplemente asociará el ID del JSON con la nueva subclase lógica.
3. **Arquitectura de Nodos para Mapas:** El uso de un sistema de coordenadas espaciales independiente (Grid/Waypoints) permitirá diseñar nuevos mapas en editores externos (como *Tiled Map Editor*) e importarlos dinámicamente sin alterar la lógica de movimiento.

7. Riesgos Técnicos y Mitigación

1. **Riesgo: Caída de Rendimiento (FPS Drop) por exceso de objetos.** Pygame renderiza en CPU, no en GPU. Si hay cientos de enemigos y proyectiles, el juego se ralentizará.
 - *Mitigación:* Implementar el patrón **Object Pooling** (Agrupación de Objetos) para los proyectiles. En lugar de crear y destruir objetos en memoria constantemente (lo cual dispara el *Garbage Collector*), se reutilizará un conjunto de proyectiles pre-creados.
2. **Riesgo: Cuellos de botella en la detección de colisiones.** Comprobar distancias entre cada torre y cada enemigo en cada *frame* es una operación de complejidad $O(N*M)$.
 - *Mitigación:* Usar las funciones nativas de optimización de Pygame (`pygame.sprite.spritecollide`) y dividir el mapa en zonas, de modo que una torre solo calcule distancias con los enemigos que se encuentren en su mismo cuadrante temporal.
3. **Riesgo: Código Espagueti en la Interfaz (UI).** Es común mezclar la lógica de dibujo de botones con la lógica del juego.

- *Mitigación:* Separar estrictamente el MVC (Modelo-Vista-Controlador). La UI solo debe leer variables del ResourceManager o reaccionar a eventos, nunca modificar la lógica del juego directamente.

8. Recomendaciones Profesionales

- **Metodologías Ágiles:** Dado el contexto de investigación, te sugiero enfocar el desarrollo en sprints cortos. Comienza con un Producto Mínimo Viable (MVP) que solo tenga un cuadro blanco que dispara a un cuadro rojo moviéndose en línea recta, y a partir de ahí itera añadiendo el arte y las métricas.
- **Modelado UML:** Antes de escribir la primera línea de código, te recomiendo levantar diagramas UML de clases y de secuencia para los flujos más complejos (como el cálculo de daño de una torre hacia un enemigo). Esto dará un respaldo académico muy robusto a la comparación de las IA.
- **Type Hinting:** Usa tipado estático en Python (módulo typing). Facilita enormemente el autocompletado en Visual Studio Code y reduce errores de paso de parámetros, algo crucial cuando se compara código generado por IA.
- **Entornos Virtuales:** Crea un entorno virtual (python -m venv venv) para aislar la versión de Pygame y asegurar que cualquier evaluador del proyecto reproduzca exactamente las mismas condiciones.

Esta arquitectura establece una base técnica robusta, preparada para evolucionar y ser evaluada críticamente.